# 0198. House Robber

n | Category: 1-D Dynamic Programming | LeetCode: https://leetcode.com/problem

# Problem

- Maximise money robbed from a row of houses without robbing two adjacent houses.

# Intuition

- At house i: either skip it (keep best up to i-1) or rob it (best up to i-2 plus nums[i]).
- Dp[i] = max(dp[i-1], dp[i-2] + nums[i]).

# Brute Force

- Try all non-adjacent subsets: O(2^n).

# Optimal Approach

- Prev2 = 0, prev1 = 0.
- For x: cur = max(prev1, prev2 + x); prev2 = prev1; prev1 = cur.

# Worked Example

```
[2,7,9,3,1].

x=2 best=2
x=7 best=max(2, 0+7)=7
x=9 best=max(7, 2+9)=11
x=3 best=max(11, 7+3)=11
x=1 best=max(11, 11+1)=12
```

# Java Solution

```java
public class HouseRobber {
    public int rob(int[] nums) { return rob(nums, 0, nums.length - 1); }

    public int rob(int[] nums, int lo, int hi) {
        int prev2 = 0, prev1 = 0;
        for (int i = lo; i <= hi; i++) {
            int cur = Math.max(prev1, prev2 + nums[i]);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

# Complexity

- Time: O(n). Space: O(1).

# Pitfalls

- Greedy "take every other house" fails: [2,1,1,2] → 4.